

AI TRAINING – DAY 6 ASSIGNMENT

Python Functions: Define → Pass → Return → Reuse

Learning Guidance + Refactoring Practice + Function-Based Utility Module

Level	Beginner → Medium
Suggested Time	3–4 hours
Core Topics	functions, parameters, return values, default arguments, *args, **kwargs
Daily Deliverable	Function-Based Utility Module
Main Focus	Turn repeated code into reusable functions

Important: What Is the Student Deliverable?

This assignment is your learning guidance. You do not need to submit a separate file for every activity. Your final Day 6 submission is ONE notebook containing your learning and testing, plus ONE Python utility module containing the final reusable functions.

```
Day_06_Function_Practice_YourName.ipynb
```

```
day_06_utility_module.py
```

Learning Goals

- Understand why functions are used in programming.
- Explain functions, parameters and return values in your own words.
- Create functions and call them correctly.
- Pass information using parameters.
- Return results using return.
- Use default arguments.
- Understand and use *args.
- Understand and use **kwargs.
- Identify common function errors and improve the solution.
- Refactor earlier loop programs into reusable functions.

How to Use This Assignment

1. Read and understand the concept before coding.
2. Write a small example before moving to the next concept.
3. Predict what the function will return before running it.
4. Try changing the arguments and observe the result.
5. Refactor repeated code instead of copying and pasting.
6. If you use AI for help, understand and explain the final solution yourself.
7. Keep your final reusable functions in the utility module.

PART A – Understand Functions

1. What is a Function?

Think about: Why put code inside a function instead of writing it every time?

```
def greet():
    print("Hello!")

greet()
greet()
```

Explain: What is the function? What is the function call? How many times does it execute?

greet() is the function. greet() at the bottom is the function call. It executes 2 times. We use functions so we don't have to type the exact same code over and over.

2. Function with a Parameter

```
def greet(name):
    print("Hello", name)

greet("Ravi")
greet("Anita")
```

Experiment by changing the values passed to name.

The parameter is name. Changing the name passed into it changes who gets greeted.

3. Return Value

```
def add(a, b):
    return a + b

result = add(10, 20)
print(result)
```

Explain the difference between print and return.

print just shows the result on the screen for the human. return gives the result back to the program so the computer can use it later in a calculation.

4. Multiple Parameters

```
def calculate_total(price, quantity):
    return price * quantity

print(calculate_total(100, 3))
```

Explain what each parameter represents.

price represents the cost of one single item. quantity represents how many of that item you are buying.

PART B – Parameters vs Arguments

- Parameter: the variable written in the function definition.
- Argument: the actual value passed when calling the function.

```
def introduce(name, age):
    return f"{name} is {age} years old"

introduce("Rahul", 21)
```

Write three examples of your own and label parameters and arguments.

PART C – Default Arguments

```
def greet(name, city="Hyderabad"):\n    return f"{name} lives in {city}"\n\nprint(greet("Ravi"))\nprint(greet("Anita", "Bangalore"))
```

- What happens when city is not provided?
- What happens when city is provided?
- Why are default arguments useful?

Create at least 3 functions using default arguments.

1. Addition Example

```
def add(x, y):\n    return x + y\n\nadd(10, 5)
```

- **Parameters:** x and y (the empty variables waiting for data).
- **Arguments:** 10 and 5 (the real numbers you actually pass in).

2. Greeting Example

```
def welcome(guest):\n    return "Welcome " + guest\n\nwelcome("Lasya")
```

- **Parameter:** guest (the placeholder in the definition).
- **Argument:** "Lasya" (the actual name provided during the call).

3. Score Example

```
def set_score(points):\n    return points * 2\n\nset_score(50)
```

- **Parameter:** points (the variable written in the brackets).
- **Argument:** 50 (the actual value given to the function).

PART D – *args

Understand *args as a way to receive a variable number of positional arguments.

```
def add_all(*numbers):\n    total = 0\n    for number in numbers:\n        total += number\n    return total\n\nprint(add_all(10, 20))\nprint(add_all(10, 20, 30, 40))
```

Experiment with 0, 1, 2 and many arguments. Explain what *args gives you inside the function.

0 arguments

```
print(add_all())
```

Output: 0

1 argument

```

print(add_all(5))          # Output: 5

# 2 arguments

print(add_all(10, 20))     # Output: 30

# Many arguments

print(add_all(1, 2, 3, 4, 5)) # Output: 15

```

***args** acts like a flexible, expanding bag. It takes however many values you pass in (even zero) and bundles them together into a single collection (called a tuple). This allows your function to easily loop through all of them without crashing, no matter how many arguments you provide.

PART E – **kwargs

Understand ****kwargs** as a way to receive a variable number of keyword arguments.

```

def show_profile(**details):
    for key, value in details.items():
        print(key, ":", value)

show_profile(name="Ravi", age=21, city="Hyderabad")

```

Add/remove profile details and explain what data you receive inside the function.

```

# Removing details (fewer arguments)

show_profile(name="Lasya") # Output: name : Lasya

# Adding new details (more arguments)

show_profile(name="Lasya", role="Student", hobby="Coding")

# Output:      name : Lasya
              role : Student
              hobby : Coding

```

****kwargs** gives you a dictionary (a collection of key-value pairs). Whenever you pass a label with an equal sign (like `hobby="Coding"`), ****kwargs** grabs the label and the value, and bundles them safely together. This allows you to pass any extra, unplanned labeled information into the function without causing an error.

PART F – Combine the Concepts

```

def create_profile(name, city="Hyderabad", *skills, **details):
    print("Name:", name)
    print("City:", city)
    print("Skills:", skills)
    print("Other Details:", details)

create_profile("Ravi", "Bangalore", "Python", "React", age=21, college="ABC Engineering College")

```

Modify this example and explain what each part receives.

```

create_profile("Lasya", "Chennai", "Java", "SQL", role="Developer", experience="2 years")

```

What each part receives:

- name receives "Lasya" (The required first argument).
- city receives "Chennai" (This replaces the default "Hyderabad").
- *skills receives ('Java', 'SQL') (All extra un-labeled items are grouped together here).
- **details receives {'role': 'Developer', 'experience': '2 years'} (All extra labeled items are grouped together here).

PART G – Predict Before You Run

Predict the output first, then run each program.

```
def square(number):
    return number * number

x = square(5)
print(x)

def greet(name="Student"):
    return "Hello " + name

print(greet())
print(greet("Phani"))

def total(*numbers):
    return sum(numbers)

print(total(1, 2, 3))
print(total(10, 20, 30, 40))

def profile(**data):
    return data

result = profile(name="Ravi", age=21)
print(result["name"])
```

For each: Inputs → Function execution → Return value → Final output.

1. Square Program

- **Inputs:** 5
- **Function execution:** Multiplies 5 * 5
- **Return value:** 25
- **Final output:** 25

2. Greet Program (Two calls)

- **Call 1:**
 - **Inputs:** None (uses default "Student")
 - **Function execution:** Joins "Hello " and "Student"
 - **Return value:** "Hello Student"
 - **Final output:** Hello Student
- **Call 2:**
 - **Inputs:** "Phani"
 - **Function execution:** Joins "Hello " and "Phani"
 - **Return value:** "Hello Phani"
 - **Final output:** Hello Phani

3. Total Program (Two calls)

- **Call 1:**
 - **Inputs:** 1, 2, 3
 - **Function execution:** Adds 1 + 2 + 3 using sum()
 - **Return value:** 6

- **Final output:** 6
- **Call 2:**
 - **Inputs:** 10, 20, 30, 40
 - **Function execution:** Adds 10 + 20 + 30 + 40 using sum()
 - **Return value:** 100
 - **Final output:** 100

4. Profile Program

- **Inputs:** name="Ravi", age=21
- **Function execution:** Packages them into a dictionary
- **Return value:** {'name': 'Ravi', 'age': 21}
- **Final output:** Ravi (because the print statement specifically asked for result["name"])

PART H – Common Function Errors

For every bug: identify the problem, explain why it happens, fix it and test it.

Bug 1

```
def add(a, b)
    return a + b
```

What is missing?

- **Problem/Explanation:** The colon (:) is missing at the end of the first line. Python needs it to know a code block is starting.
- **Fix & Test:**

```
def add(a, b):
    return a + b

print(add(5, 3)) # Output: 8
```

Bug 2

```
def add(a, b):
    print(a + b)

result = add(10, 20)
print(result)
```

Explain print vs return.

- **Problem/Explanation:** print just shows the answer on the screen, but it returns nothing back to the code. Because of this, the result variable becomes None. return actually hands the value back so you can save it.
- **Fix & Test:**

```
def add(a, b):
    return a + b

result = add(10, 20)

print(result) # Output: 30
```

Bug 3

```
def greet(name):  
    return "Hello " + name  
  
print(greet())
```

Why does this fail? Fix it using a default or supplied argument.

- **Problem/Explanation:** The function fails because it expects exactly one argument (name), but you called it with empty brackets ().
- **Fix & Test (using default):**

```
def greet(name="Guest"):  
    return "Hello " + name  
  
print(greet()) # Output: Hello Guest
```

Bug 4

```
def calculate(price, quantity=1):  
    return price * quantity  
  
print(calculate())
```

Why does this still fail?

- **Problem/Explanation:** It still fails because price does not have a default value. Even though quantity defaults to 1, you still have to give it a price.
- **Fix & Test:**

```
def calculate(price, quantity=1):  
    return price * quantity  
  
print(calculate(100)) # Output: 100
```

Bug 5

```
def add_all(*numbers):  
    return numbers + 10
```

What is numbers? Fix the function.

- **Problem/Explanation:** numbers is a tuple (a grouped collection of numbers), not a single number. You cannot mathematically add the number 10 to a tuple. You need to sum the collection first.
- **Fix & Test:**

```
def add_all(*numbers):  
    return sum(numbers) + 10  
  
print(add_all(5, 5)) # Output: 20
```

Bug 6

```
def show(**details):  
    print(details)
```

```
show("Ravi", 21)
```

Why is this not a correct use of ****kwargs**? Fix the call.

- **Problem/Explanation:** ****details** (****kwargs**) only accepts labeled keyword arguments (like `name="Ravi"`). Sending `"Ravi"` by itself without a label breaks it.
- **Fix & Test:**

```
def show(**details):
```

```
    print(details)
```

```
show(name="Ravi", age=21)          # Output: {'name': 'Ravi', 'age': 21}
```

PART I – Refactor Previous Loop Programs

Take your Day 5 loop programs and convert them into reusable functions. Refactor at least:

8. Multiplication table
9. Sum of numbers
10. Prime number checker
11. Palindrome checker
12. Find factors
13. Pattern printing

Example:

```
# Before
number = 7
for i in range(1, 11):
    print(number * i)

# After
def multiplication_table(number):
    for i in range(1, 11):
        print(number * i)

multiplication_table(7)
multiplication_table(12)
```

Do not merely put old code inside `def`. Ask: What should the function receive? What should it return? Can another program reuse it?

1. Multiplication Table

- **Receives:** A number to multiply.
- **Returns:** Nothing (it just prints the table directly).

```
# Before
```

```
number = 5
```

```
for i in range(1, 11):
```

```
    print(number, "x", i, "=", number * i)
```

```
# After

def multiplication_table(number):

    for i in range(1, 11):

        print(number, "x", i, "=", number * i)

multiplication_table(5)
```

2. Sum of Numbers

- **Receives:** A number n.
- **Returns:** The final total (so other programs can use the math result).

```
# Before

n = 5

total = 0

for i in range(1, n + 1):

    total = total + i

print(total)
```

```
# After

def calculate_sum(n):

    total = 0

    for i in range(1, n + 1):

        total = total + i

    return total

print(calculate_sum(5))
```

3. Prime Number Checker

- **Receives:** A number to check.
- **Returns:** True if it is prime, False if it is not.

```
# Before

num = 7

is_prime = True

for i in range(2, num):

    if num % i == 0:

        is_prime = False

        break

print(is_prime)
```

```
# After

def is_prime(number):
    if number <= 1:
        return False

    for i in range(2, number):
        if number % i == 0:
            return False

    return True

print(is_prime(7))
```

4. Palindrome Checker

- **Receives:** A number.
- **Returns:** True if it reads the same backwards, False if not.

```
# Before

num = 121

original = num

reversed_num = 0

while num > 0:
    reversed_num = (reversed_num * 10) + (num % 10)
    num = num // 10

print(original == reversed_num)
```

```
# After

def is_palindrome(number):
    original = number
    reversed_num = 0

    while number > 0:
        reversed_num = (reversed_num * 10) + (number % 10)
        number = number // 10

    return original == reversed_num

print(is_palindrome(121))
```

5. Find Factors

- **Receives:** A number.

- **Returns:** A list of all its factors (instead of printing them, returning a list makes it reusable).

Before

```
num = 12

for i in range(1, num + 1):
    if num % i == 0:
        print(i)
```

After

```
def find_factors(number):
    factors = []

    for i in range(1, number + 1):
        if number % i == 0:
            factors.append(i) # Add to the list instead of printing

    return factors

print(find_factors(12))
```

6. Pattern Printing

- **Receives:** The number of rows.
- **Returns:** Nothing (it just prints the visual shape to the screen).

Before

```
rows = 3

for i in range(1, rows + 1):
    print("*" * i)
```

After

```
def print_pattern(rows):
    for i in range(1, rows + 1):
        print("*" * i)

print_pattern(3)
print_pattern(5)
```

PART J – Function Practice Problems

Problem 1 – Temperature Converter

Create functions for Celsius → Fahrenheit and Fahrenheit → Celsius.

```
def to_fahrenheit(celsius):
    return (celsius * 9/5) + 32

def to_celsius(fahrenheit):
    return (fahrenheit - 32) * 5/9

print(to_fahrenheit(0))    # Output: 32.0
print(to_celsius(100))    # Output: 37.77
```

Problem 2 – Even/Odd Function

Create a function that receives a number and returns whether it is even or odd.

```
def check_even_odd(number):
    if number % 2 == 0:
        return "Even"
    else:
        return "Odd"

print(check_even_odd(10)) # Output: Even
print(check_even_odd(7))  # Output: Odd
```

Problem 3 – Prime Function

Create reusable is_prime(number) returning True or False.

```
def is_prime(number):
    if number <= 1:
        return False
    for i in range(2, number):
        if number % i == 0:
            return False
    return True

print(is_prime(7))    # Output: True
print(is_prime(10))   # Output: False
```

Problem 4 – Palindrome Function

Create reusable is_palindrome(value).

```
def is_palindrome(value):
    word = str(value)
    return word == word[::-1]

print(is_palindrome(121))    # Output: True
print(is_palindrome("madam")) # Output: True
```

```
print(is_palindrome(123))      # Output: False
```

Problem 5 – Grade Function

Create a function that receives marks and returns a grade; handle invalid marks.

```
def calculate_grade(marks):  
    if marks < 0 or marks > 100:  
        return "Invalid Marks"  
  
    elif marks >= 90:  
        return "A"  
  
    elif marks >= 75:  
        return "B"  
  
    elif marks >= 50:  
        return "C"  
  
    else:  
        return "Fail"  
  
print(calculate_grade(85))    # Output: B  
print(calculate_grade(105))  # Output: Invalid Marks
```

Problem 6 – Shopping Total

Create a function receiving price and quantity and returning the total.

```
def calculate_total(price, quantity):  
    return price * quantity  
  
print(calculate_total(50, 3)) # Output: 150
```

Problem 7 – Flexible Sum

Create add_all(*numbers) returning the sum of any number of values.

```
def add_all(*numbers):  
    return sum(numbers)  
  
print(add_all(10, 20, 30)) # Output: 60  
print(add_all(5, 5))       # Output: 10
```

Problem 8 – Profile Builder

Create build_profile(**details) returning a profile dictionary.

```
def build_profile(**details):  
    return details  
  
print(build_profile(name="Lasya", age=21))  
  
# Output: {'name': 'Lasya', 'age': 21}
```

Problem 9 – Greeting Utility

Create a greeting function with a default city and optional name.

```
def greet(name="Guest", city="Hyderabad"):

    return "Hello " + name + ", welcome to " + city

print(greet()) # Output: Hello Guest, welcome to Hyderabad

print(greet("Lasya", "Bangalore")) # Output: Hello Lasya, welcome to Bangalore
```

Problem 10 – Statistics Utility

Create functions for minimum, maximum and average of a collection of numbers.

```
def get_statistics(*numbers):

    minimum = min(numbers)

    maximum = max(numbers)

    average = sum(numbers) / len(numbers)

    return minimum, maximum, average

print(get_statistics(10, 20, 30))

# Output: (10, 30, 20.0)
```

PART K – Main Project: Function-Based Utility Module

Create one reusable Python utility module, like a small library another Python program could import.

Your module must contain at least 10 reusable functions. Suggested functions:

- is_even(number)
- is_prime(number)
- is_palindrome(value)
- calculate_sum(numbers) or add_all(*numbers)
- calculate_average(numbers)
- find_factors(number)
- calculate_grade(marks)
- calculate_total(price, quantity=1)
- celsius_to_fahrenheit(celsius)
- fahrenheit_to_celsius(fahrenheit)

You may replace functions with your own useful utilities, but the module must demonstrate all Day 6 concepts.

Required Function Concepts in the Final Module

- At least 10 reusable functions.
- Functions with normal parameters.
- Functions that return values.
- At least 2 functions with default arguments.
- At least 1 function using *args.
- At least 1 function using **kwargs.
- At least 3 functions refactored from Day 5 loop programs.

- Input validation where appropriate.
- Clear and meaningful function names.

Example Module Usage

```
from day_06_utility_module import is_prime, calculate_grade

print(is_prime(17))
print(calculate_grade(82))
```

Your module should work when imported from another Python file.

Module:

1. Normal parameter (Refactored from Day 5)

```
def is_even(number):

    return number % 2 == 0
```

2. Normal parameter (Refactored from Day 5)

```
def is_prime(number):

    if number <= 1:

        return False

    for i in range(2, number):

        if number % i == 0:

            return False

    return True
```

3. Normal parameter (Refactored from Day 5)

```
def is_palindrome(value):

    word = str(value)

    return word == word[::-1]
```

4. Normal parameter (Returns a list)

```
def find_factors(number):

    factors = []

    for i in range(1, number + 1):

        if number % i == 0:

            factors.append(i)

    return factors
```

5. Input validation included

```
def calculate_grade(marks):
```

```

    if marks < 0 or marks > 100:

        return "Invalid marks"

    elif marks >= 90:

        return "A"

    elif marks >= 75:

        return "B"

    elif marks >= 50:

        return "C"

    else:

        return "Fail"

# 6. Default argument 1

def calculate_total(price, quantity=1):

    return price * quantity

# 7. Default argument 2

def greet_user(name, greeting="Welcome"):

    return greeting + " " + name

# 8. Math function

def celsius_to_fahrenheit(celsius):

    return (celsius * 9/5) + 32

# 9. *args function

def calculate_sum(*numbers):

    return sum(numbers)

# 10. **kwargs function

def build_profile(**details):

    return details

```

Example Module Usage:

```

from day_06_utility_module import *

print("Is 17 prime?", is_prime(17))

print("Grade for 82:", calculate_grade(82))

print("Grade for 105:", calculate_grade(105)) # Testing validation

print("Total for 1 item:", calculate_total(50)) # Uses default quantity 1

print("Greeting:", greet_user("Lasya")) # Uses default greeting

```

```
print("Sum of numbers:", calculate_sum(10, 20, 30, 40)) # Testing *args

print("Profile details:", build_profile(name="Lasya", role="Student", age=21)) # Testing
**kwargs
```

OUTPUT:

```
Is 17 prime? True

Grade for 82: B

Grade for 105: Invalid marks

Total for 1 item: 50

Greeting: Welcome Lasya

Sum of numbers: 100

Profile details: {'name': 'Lasya', 'role': 'Student', 'age': 21}
```

PART L – Testing Your Functions

For important functions, test normal, boundary, invalid and different argument combinations where relevant.

- Normal input.
- Boundary input.
- Invalid input where applicable.
- Small and large values where relevant.
- Different combinations of arguments.

Create a small test section at the bottom of your notebook.

```
# 1. Normal Input

print("Normal (Prime):", is_prime(7))

print("Normal (Grade):", calculate_grade(85))

# 2. Boundary Input (Testing the exact edge numbers)

print("Boundary (Lowest Grade):", calculate_grade(0))

print("Boundary (Highest Grade):", calculate_grade(100))

# 3. Invalid Input (Testing numbers that shouldn't work)

print("Invalid (Negative Grade):", calculate_grade(-10))

print("Invalid (Over 100 Grade):", calculate_grade(105))

# 4. Different Combinations (Testing default arguments)

print("Combo (Default Quantity):", calculate_total(50))

print("Combo (Custom Quantity):", calculate_total(50, 3))

# 5. Small and Large Values (Testing *args flexibility)

print("Small (One number):", calculate_sum(5))

print("Large (Many numbers):", calculate_sum(1000, 2000, 3000, 4000))
```

OUTPUT:

```
Normal (Prime): True  
Normal (Grade): B  
Boundary (Lowest Grade): Fail  
Boundary (Highest Grade): A  
Invalid (Negative Grade): Invalid marks  
Invalid (Over 100 Grade): Invalid marks  
Combo (Default Quantity): 50  
Combo (Custom Quantity): 150  
Small (One number): 5  
Large (Many numbers): 10000
```

PART M – Explain Your Code

14. Why are functions useful?

They stop you from typing the exact same code twice. You write the logic once and can reuse it anywhere.

15. What is the difference between a parameter and an argument?

A parameter is the empty placeholder variable when you create the function. An argument is the real data you pass in when you actually run it.

16. What is the difference between print and return?

print just displays the result on the screen for the user to see. return hands the result back to the code so the computer can use it in the next step.

17. Why use a default argument?

It saves time. It automatically fills in a standard value (like quantity=1) if the user doesn't provide one.

18. What does *args allow you to do?

It lets your function accept as many regular arguments as you want (like a list of numbers) without causing an error.

19. What does **kwargs allow you to do?

It lets your function accept as many labeled arguments as you want (like name="Lasya", age=21) without causing an error.

20. Why is refactoring useful?

It cleans up messy, repeated code and turns it into neat, reusable blocks that other programs can easily import and use.

21. Which Day 5 program became easier to reuse after conversion?

The Prime Number checker. Now, instead of writing a whole loop every time, I can just type is_prime(7) and get the answer instantly.

22. Which function was hardest to design?

The Statistics Utility. It was a bit tricky to figure out how to calculate and return the minimum, maximum, and average all at the exact same time.

23. What bug did you face and how did you improve your solution?

I accidentally used print instead of return in my math functions, which made my answer equal None. I fixed it by making sure every function handed the math back using the return keyword.

PART N – STUDENT DELIVERABLE – WHAT YOU MUST SUBMIT

Submit TWO files. One notebook for your learning/practice and one Python module for your final reusable functions.

```
Day_06_Function_Practice_YourName.ipynb
```

```
day_06_utility_module.py
```

The notebook must contain:

24. Your own explanations of functions, parameters and return values.
25. Default argument practice.
26. *args practice.
27. **kwargs practice.
28. Output-prediction exercises.
29. Debugging exercises and corrected solutions.
30. Solutions to the 10 function practice problems.
31. Refactoring notes for Day 5 programs.
32. Testing of your utility functions.
33. Final reflection.

The Python utility module must contain:

34. At least 10 reusable functions.
35. Normal parameters and return values.
36. At least 2 default-argument functions.
37. At least 1 *args function.
38. At least 1 **kwargs function.
39. At least 3 functions refactored from Day 5 programs.
40. Meaningful names and readable code.

You DO NOT Need to Submit

- A separate file for every function.
- A separate file for every practice problem.
- Screenshots of every output.
- A separate Word/PDF document.

Submission Checklist

- ☒ Functions explained in my own words.
- ☒ Parameters and arguments practiced.
- ☒ Return values practiced.
- ☒ Default arguments practiced.
- ☒ *args practiced.
- ☒ **kwargs practiced.
- ☒ Debugging completed.
- ☒ 10 function problems completed.
- ☒ Previous loop programs refactored.
- ☒ Utility module contains at least 10 functions.
- ☒ Utility module contains default args.
- ☒ Utility module contains *args.

- ☒ Utility module contains `**kwargs`.
- ☒ Functions tested.
- ☒ Both files are ready to submit.

Suggested Evaluation

Area	Weight
Concept understanding + explanation	15%
10 Function Problems	25%
Refactoring Day 5 Programs	20%
Utility Module	25%
Testing + Debugging	10%
Code Quality + Reflection	5%

DAY 6 PRINCIPLE

Don't write the same logic again and again. Write it once, make it reusable, and call it whenever you need it.